Project Report: Week 2 Summary & Technical Revision

1. Fundamentals of Technical Analysis

- **Price Aggregation (OHLC):** Learned to summarize daily price action using four key data points: **Open** (initial execution), **High** (peak intensity), **Low** (maximum selling pressure), and **Close** (the final, most critical reference price indicating intraday sentiment).
- **Chart Abstractions:** Evaluated the simplicity of **Line Charts** against the comprehensive but visually complex **Bar Charts**, moving ultimately to **Candlestick Charts** to easily map market psychology.

---

2. Candlestick Patterns & Market Psychology

Candlesticks translate pure market emotion into actionable patterns. The core structures reviewed include:

- **Marubozu (Bullish/Bearish):** Candles with no shadows (`Open = Low` or `Open = High`). Indicates absolute dominance by one side from the opening bell to the market close.
- **Spinning Tops & Dojis:** Characterized by tiny or non-existent real bodies. They signify extreme market indecision where both bulls and bears attempt rallies but fail to hold ground.
- **Paper Umbrellas (Hammer & Hanging Man):** Features a small upper body and a long lower shadow. A **Hammer** at the bottom of a downtrend signals a bullish reversal; a **Hanging Man** at the top of an uptrend signals a bearish fatigue.
- **Engulfing Patterns & Harami:** Two-candle multi-day patterns. *Engulfing* represents a sudden trend-shattering momentum reversal, while *Harami* (parent-child structure) signifies a sudden containment of volatility pointing to an impending breakout.

---

3. Core Indicators & python Code Revision

To systematically analyze these trends, Week 2 focused on programmatic technical analysis using the `yfinance` library to download historical assets, alongside calculating key statistical indicators.

A. Fetching Market Data (`yfinance`)

Used to seamlessly fetch clean, tabular market data directly from Yahoo Finance.

```
import yfinance as yf

# 1. Instantiate the Asset Pointer
ticker = yf.Ticker("AAPL")

# 2. Extract Historical Time-Series
historical_data = ticker.history(period="1y")

# 3. Isolate Stock Actions
dividends_splits = ticker.actions
```

Code Term Breakdown:

- `yf.Ticker("AAPL")`: Creates a localized core Python object targeting a specific market ticker symbol (e.g., Apple Inc.).
- `.history(period="1y")`: Fetches a structural Pandas DataFrame containing historical daily OHLCV rows for the designated time frame (`"1y"` for 1 year, `"1mo"` for 1 month).
- `.actions`: Automatically extracts a standalone DataFrame tracking corporate events, specifically historical stock splits and dividend payouts.

B. Moving Averages (SMA vs. EMA)

Moving averages smooth out daily price noise to reveal the underlying macro trend.

```
# 1. Simple Moving Average (SMA)
df['SMA_50'] = df['Close'].rolling(window=50).mean()

# 2. Exponential Moving Average (EMA)
```

```
df['EMA_50'] = df['Close'].ewm(span=50, adjust=False).mean
()
```

Code Term Breakdown:

- `.rolling(window=50)`: Sets up a sliding window execution context over the last 50 consecutive rows of historical data.
- `.mean()`: Computes a standard arithmetic average across that rolling window. It drops the oldest price as a new day arrives.
- `.ewm(span=50, adjust=False)`: Invokes an **Exponentially Weighted Moving** calculation. Unlike the regular SMA, the EMA applies a multiplier to give significantly more weight to the most recent price points, making it stick closer to the current market trend and react faster to sharp reversals.

C. Advanced Oscillators & Bands (MACD & Bollinger Bands)

Used to calculate momentum and statistical volatility boundaries.

```
# 1. MACD Spread Calculation
ema_12 = df['Close'].ewm(span=12, adjust=False).mean()
ema_26 = df['Close'].ewm(span=26, adjust=False).mean()
df['MACD_Line'] = ema_12 - ema_26

# 2. Bollinger Bands Volatility Channels
df['Middle_Band'] = df['Close'].rolling(window=20).mean()
df['Standard_Deviation'] = df['Close'].rolling(window=20).s
td()
df['Upper_Band'] = df['Middle_Band'] + (2 * df['Standard_De
viation'])
df['Lower_Band'] = df['Middle_Band'] - (2 * df['Standard_De
viation'])
```

Code Term Breakdown:

- `ema_12 - ema_26`: The **MACD Line** represents the structural convergence or divergence of short-term vs. long-term momentum. Crossing above the zero centerline indicates accelerating bullish momentum.

- `.std()`: Computes the **Standard Deviation** over a 20-day window, mathematically measuring the asset's current volatility.
- `+ (2 * df['Standard_Deviation'])`: Establishes an outer statistical channel boundary. Statistically, prices should hover inside these bands. Reaching the **Upper_Band** suggests the asset is temporarily overbought (expensive), while touching the **Lower_Band** implies it is oversold (cheap), triggering a mean-reversion expectation.

---

4. Market Phases & The Grand Checklist

- **Dow Theory Tenets:** Established that indices discount everything, volume must confirm price movements, and closing prices are the most sacred data points.
- **Market Phases:** Highlighted how markets move in repeating cycles: **Accumulation** (smart money quietly buying at the bottom), **Markup** (rapid vertical price acceleration), and **Distribution** (retail FOMO at the top while smart money exits).
- **The Grand Checklist:** Formulated a 6-step confirmation framework required before executing any strategic trade:
    1. Identifiable Candlestick Pattern.
    2. Support & Resistance confirmation (defining stop-loss parameters).
    3. Above-average Volume confirmation.
    4. Dow Theory perspective (aligning with primary/secondary trends).
    5. Indicator alignment (EMA Crossovers, RSI oversold/overbought states).
    6. A strict **Reward to Risk Ratio (RRR)** threshold of at least 1.5.